

Monash University
FIT5216 Modelling discrete optimisation problems - S1 2025
Assignment 3:
Service Scheduling Report

Name: Rishabh Ray
Student ID: 34525416

Stage G: Report – Analysis of Input Assumptions and Their Effects

minservice = 0

When I set minservice = 0, I gave my model complete freedom as there was no obligation to deliver any minimum number of services. What I observed was that the model chose only those assignments that would minimise the objective, often leaving some trucks unused and many services unassigned. For example, in some runs, only a few trucks like T1 and T3 were assigned any jobs, with the rest left idle. This resulted in very low objective values, sometimes dropping below 2000, but only a small number of services were completed. It showed that when no external constraints are applied, the model's natural behaviour is to "do less but do it efficiently," focusing only on what is cheapest and ignoring the rest. My outputs reflected this minimal approach, with very light workloads and plenty of unused capacity.

minservice = 100

Once I changed minservice to 100, I saw an immediate shift. Now, the model was forced to deliver every single job. This strict requirement pushed the model to use more trucks and schedule jobs even if it led to higher overlap in maintenance and higher objective values, often exceeding 30,000 in my solutions. I noticed every truck (except for the dummy) was used, and the schedule became denser and more complex, with more frequent maintenance and less idle time per truck. My outputs showed that all 10 services were always covered, but at the cost of more maintenance and higher operational costs. The minimum service requirement became the main driver, reducing flexibility and driving up the objective, making the problem much more challenging for the solver.

minwork = 0

Setting minwork to zero meant the model could use trucks as little as it liked. In my outputs, I saw trucks with only one or two short assignments, as the model sought to meet the overall service minimum but avoided overusing any single truck. For instance, some solutions had as many as five trucks used, but with several only working for a single time slot. This made the assignments quite fragmented and often led to low objective values. The model sometimes covered 8 or 9 services, but no truck was fully loaded. The main constraint now was simply covering enough services, with very little concern for whether trucks were used efficiently, and my objective values reflected this by staying quite low, around 5000 in several cases.

minwork = 100

When I pushed minwork up to 100, meaning every used truck had to be fully loaded across the entire schedule, I found that the model couldn't find any feasible solution with my data. There just wasn't any way to fit jobs so that every working truck was used for exactly 15 time units. I got unsatisfiable solutions, with the solver failing to assign enough services to meet both the minwork and minservice thresholds. This taught me that too-strict workload requirements can quickly make the whole problem unsolvable, especially when job durations and time windows don't add up neatly to the scheduling horizon.

maxsty = [1,1,1,0]

When I limited each truck to just one Gas, one Refit, and one Roof job, but allowed unlimited Solar, I saw flexibility disappear. The trucks were only given one or two jobs, and jobs of the unconstrained type (Solar) piled up on certain trucks. I observed that in my outputs, some trucks were assigned just a single service, and the total number of assigned jobs rarely exceeded half the available capacity. This resulted in higher objective values, sometimes in the range of 15,000 or more, as the model had to spread jobs out inefficiently. It became obvious to me that strict type caps force the model to spread jobs across more trucks and leave a lot of capacity unused, leading to increased idle time and higher operational costs.

maintdur[SH] > maintdur[LG]

In this case, when I swapped the durations so that SH maintenance took longer than LG, I noticed that the model started treating SH as much more disruptive. My solutions scheduled SH maintenance less often, and with more care, to avoid conflicts. I observed that trucks with more SH maintenance had longer gaps between jobs, and there were often fewer overall SH events in my output. This increase in maintenance duration directly raised my objective values and forced the model to spread jobs further apart, which sometimes resulted in underutilised trucks or unassigned services.

maxsep[SH] > maxsep[LG]

When I set a larger separation for SH maintenances compared to LG, the model was forced to space SH events much further apart. In the output, I could clearly see SH maintenances were fewer and more spread out, sometimes with only one or two per truck for the whole period. Objective values tended to rise, as the model had fewer opportunities to fit SH events into the schedule, often leading to gaps and missed services. The constraint on separation took over, and I watched as the model chose when and where to insert SH maintenances very selectively, sometimes even skipping them altogether if possible.

maxsep[MJ] > maxsep[LG]

Making MJ (major) maintenance the rarest, by setting its separation highest, led to MJ events being scattered far apart in my schedules, with more frequent LG and SH maintenances filling in the gaps. I noticed that trucks would often only see a single MJ event across the entire schedule, and other types of maintenance were used to fill in requirements. Objective values tended to stabilize or drop, as the infrequent MJ events reduced disruption, and I saw more services completed with less maintenance overlap. This setup showed that maxsep[MJ] acts as a pacing mechanism for the whole maintenance schedule.

maxsep[SH] = maxsep[LG] = endtime

When I set both SH and LG separations to the scheduling horizon, I noticed these maintenance almost vanished from the solution. With this change, minimum service and work requirements became the key influencers. My outputs had very few maintenance periods, and the objective values were often the lowest I observed, sometimes below 1000, as the model used the freed-up time for service delivery.

prework = [0 | t in TRUCK]

Setting all trucks' prework to zero meant every truck started as "freshly maintained." I saw that this allowed the model to use trucks to their full potential right from the start, with no forced early maintenance. My results showed more front-loaded schedules, with jobs assigned earlier and maintenance pushed toward the end or sometimes skipped altogether. The objective values reflected efficient use, and the number of jobs completed per truck was higher than in runs with higher prework.

majorwork > endtime

By increasing majorwork to a value greater than my scheduling horizon, I found that no truck ever hit the limit for required major maintenance. This essentially turned off the majorwork constraint, and the model's decisions focused entirely on the other requirements—mainly minimum work and service. The outputs showed fewer maintenance breaks and more continuous service, and my objective values were usually lower, showing that the trucks could be used more flexibly.

maxsty[ty] = 0 for some ty other than DUMMY

In this assignment, the maxsty array determines the maximum number of services of each type that can be performed during the scheduling period. When maxsty[ty] = 0 for a particular service type ty (such as Roof), it completely prohibits any truck from performing that type of service in the current schedule, regardless of availability or other constraints. This setting has a strong impact: even if there are scheduled services of that type listed in the stype array, the model will ensure that no truck is assigned to deliver those services, resulting in unallocated or unserved jobs of that type. This makes it impossible to cover these services in the plan, which can lead to not meeting the required minimum service (minservice) or work (minwork) thresholds if those jobs are needed to reach the target. It may also affect the feasibility of the solution, depending on how other constraints are set.

start[s] = 0 and end[s] = endtime for some service s

In this assignment, when I set start[s] = 0 and end[s] = endtime for a service (such as S2) means that this service is available to be scheduled at any time throughout the entire planning horizon. This gives the scheduler maximum flexibility in assigning this service to any truck and at any feasible time, making it easier to meet constraints such as minimum work (minwork) and minimum service (minservice). In the provided output, we see that services with this full-time window can be assigned continuously across all available periods (e.g., S2 is assigned to T1 and runs for all 15 units of time). This often results in at least one truck dedicating its full capacity to that service type, maximising resource usage and contributing significantly to the work and service totals. However, if multiple services have this unrestricted window, I found that it may crowd out other jobs with more restrictive time windows, so balancing these settings is important for feasible and optimal scheduling.

start includes k services with the same start and end time where $k = \text{card}(\text{TRUCK})$

When there were as many simultaneous jobs as trucks, all with the same tight time window, I saw the model assign each job to a different truck, maximising resource use during that period. In my outputs, every truck was busy at the same time, but this sometimes caused problems with other requirements, especially if skills or service type limits didn't match up. If there was a mismatch, my model often left some jobs unassigned or failed to meet minimum work, resulting in infeasibility or higher objectives. The solutions here gave a clear demonstration of the model's ability to handle peak demand periods, but also exposed where constraints could cause failure.

$\text{card}(\text{STYPE}) = 1$

When I set $\text{card}(\text{STYPE}) = 1$, meaning all services are of a single type (e.g., all Gas), the assignment model becomes less complex in terms of skill matching since every truck that is eligible for that type can be assigned to any service. However, the model must still satisfy all other assignment requirements, such as maintenance constraints, the maximum number of services per truck, minimum work and service thresholds, and valid truck-service assignments without overlaps. This scenario primarily tests the model's ability to balance workload, respect time windows, and efficiently allocate resources when only one service type is in demand, ensuring robust handling of homogeneous service environments as required by the assignment specification.

Summary: Most Important Constraints

Among all the constraints, the most influential ones I found to be are:

- **Minimum Service Requirement** - It dictates how many services must be scheduled; raising it forces more jobs to be assigned, directly increasing workload and objective value.
- **Minimum Work Requirement per Truck** - Controls truck utilisation; high values can make solutions infeasible, while low values allow flexible, fragmented assignments.
- **Maximum Services per Type** - Restricts how many jobs of each type a truck can perform; strict limits lead to underutilization and higher costs.
- **Maintenance Frequency and Separation** - Governs how often, how long, and how far apart maintenance events must be scheduled; tight constraints increase complexity and can drive up the objective.
- **Major Maintenance Threshold** - If set below the scheduling horizon, can force frequent interruptions; if above, it effectively disables a key constraint, increasing flexibility.
- **Service Time Windows** - Narrow time windows or simultaneous job demands reduce flexibility and can cause infeasibility if not matched with available resources.

Together, these constraints enforce the balance between operational efficiency and compliance with maintenance policies, with changes in any of them often having a dramatic effect on the resulting schedule.

Conclusion

Looking back, I can see how the input assumptions I chose directly shaped my model's outputs and what constraints became the most influential. My observations showed not just what the model did, but why, tying every solution back to the specific settings and demonstrating how the logic of the model adapts to different requirements. These insights gave me a real appreciation for the balance between feasibility, efficiency, and cost in practical scheduling models.